



SIMATS Online Programming Lab

JAVA PROGRAMMING


Mohammed Subahaan H
192572165RD

QUESTIONS 19 / 20 completed

Bank

BANK

Write a Java program to implement a basic banking system using inheritance. Create a parent class called Account with attributes such as account number, account holder name, and balance. Then, create child classes such as

PROGRAM EDITOR

Java

Run Save

```

1 import java.util.ArrayList;
2 import java.util.Scanner;
3
4 public class Bank {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         ArrayList<Account> accounts = new ArrayList<>();
8         String savingsAccountNumber = sc.nextLine().trim();
9         String savingsHolderName = sc.nextLine().trim();
10        double savingsInterestRate = Double.parseDouble(sc.nextLine().trim());
11        double savingsInitialDeposit = Double.parseDouble(sc.nextLine().trim());
12        accounts.add(new SavingsAccount(savingsAccountNumber, savingsHolderName, savin
13        String checkingAccountNumber = sc.nextLine().trim();
14        String checkingHolderName = sc.nextLine().trim();
15        double checkingOverdraftLimit = Double.parseDouble(sc.nextLine().trim());
16        double checkingInitialDeposit = Double.parseDouble(sc.nextLine().trim());
17        accounts.add(new CheckingAccount(checkingAccountNumber, checkingHolderName, ch
18        System.out.println("The accounts in the system are:");
19        for (int i = 0; i < accounts.size(); i++) {
20            System.out.println((i + 1) + ". " + accounts.get(i).getAccountType() +

```

INPUT

```

123456789
Jane Doe
1.5
5000
987654321
John Smith
1000
1000
1
deposit
2500
2
withdraw
2000

```



SIMATS Online Programming Lab

JAVA PROGRAMMING


Mohammed Subahaan H
192572165RD

QUESTIONS 19 / 20 completed

Employee

EMPLOYEE

Write a Java program to implement a basic employee management system using inheritance. Create a parent class called Employee with attributes such as name, id, and salary. Then, create child classes such as Manager and SalesPerson that inherit from the

PROGRAM EDITOR

Java

Run Save

```

1 public class Main {
2     public static void main(String[] args) {
3         Manager manager = new Manager("Alice", 101, 90000, 10);
4         SalesPerson salesPerson = new SalesPerson("Bob", 202, 60000, 150);
5
6         System.out.println("---- Employee Details ----");
7         System.out.println(manager.getDetails());
8         manager.manageTeam();
9
10        System.out.println();
11
12        System.out.println(salesPerson.getDetails());
13        salesPerson.makeSale(5000);
14        salesPerson.makeSale(2500);
15
16        System.out.println();
17        System.out.println("Total Sales for " + salesPerson.getName() + ": $" + salesP
18    }
19 }

```

INPUT

Your INPUT goes here!

OUTPUT

```

---- Employee Details ----
Name: Alice, ID: 101, Salary: $90000.0
Managing a team of 10 members.

Name: Bob, ID: 202, Salary: $60000.0
Bob made a sale of $5000.0. Updated total sales = $5150.0
Bob made a sale of $2500.0. Updated total sales = $7650.0

Total Sales for Bob: $7650.0

```

OUTPUT

```

---- Employee Details ----
Name: Alice, ID: 101, Salary: $90000.0
Managing a team of 10 members.

Name: Bob, ID: 202, Salary: $60000.0
Bob made a sale of $5000.0. Updated total sales = $5150.0
Bob made a sale of $2500.0. Updated total sales = $7650.0

Total Sales for Bob: $7650.0

```

QUESTIONS 19 / 20 completed

Animal-1

ANIMAL-1

Write a Java program to implement a basic animal classification system using inheritance. Create a parent class called Animal with attributes such as name and habitat. Then, create child classes such as Mammal, Reptile, and Bird that inherit from the Animal class and have their own unique

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Mammal dog = new Mammal("Dog", "Land");
4         Reptile snake = new Reptile("Snake", "Water/Land");
5         Bird eagle = new Bird("Eagle", "Sky");
6
7         System.out.println("--- Animal Classification System ---");
8
9         dog.displayInfo();
10        dog.eat();
11        dog.reproduce();
12
13        System.out.println();
14
15        snake.displayInfo();
16        snake.eat();
17        snake.reproduce();
18
19        System.out.println();
20
21        eagle.displayInfo();
```

INPUT

Your INPUT goes
here!

OUTPUT

```
--- Animal Classification System ---
Name: Dog, Habitat: Land
Dog eats milk or plants.
Dog gives birth to live young.

Name: Snake, Habitat: Water/Land
Snake eats insects or small animals.
Snake lays eggs.
```

QUESTIONS 19 / 20 completed

Shapes-1

SHAPES-1

Write a Java program to implement a basic shape hierarchy using inheritance. Create a parent class called Shape with attributes such as color and area. Then, create child classes such as Circle, Square, and Triangle that inherit from the Shape class and have their own unique methods for

PROGRAM EDITOR

Java

Run

Save

```
1 class main{
2     public static void main(String args[]){
3         Circle c=new Circle("Red",5.0);
4         Square s=new Square("Green",4.0);
5         Triangle t=new Triangle("yellow",5.0,4.0,6.0,4.5);
6         System.out.println("circle :");
7         c.display();
8         System.out.println("Square :");
9         s.display();
10        System.out.println("Triangle: ");
11        t.display();
12    }
13 }
14 class Shape{
15     String color;
16     double area;
17     public Shape(String color){
18         this.color=color;
19     }
20     void area(){
21         //to be overridden
```

INPUT

Your INPUT goes
here!

OUTPUT

```
circle :
color : Red
Area : 78.5
Radius : 5.0
Perimeter : 31.400000000000002
Square :
color : Green
Area : 16.0
Side : 4.0
Perimeter: 16.0
```

QUESTIONS 19 / 20 completed

Library

LIBRARY

Write a Java program to implement a basic library management system using inheritance. Create a parent class called Item with attributes such as title and author. Then, create child classes such as Book and DVD that inherit from the Item class and have their own unique attributes.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.*;
2 class main{
3     public static void main(String args[]){
4         Book b1=new Book("The Alchemist","paulo coelo",208);
5         DVD d1=new DVD ("Inception","Nolan",148);
6         System.out.println("Initial library items : ");
7         b1.displayInfo();
8         d1.displayInfo();
9         System.out.println("\nBorrowing items: ");
10        b1.borrowItem();
11        d1.borrowItem();
12        System.out.println("\nUpdated status: ");
13        b1.displayInfo();
14        d1.displayInfo();
15        System.out.println("\nReturning items: ");
16        b1.returnItem();
17        d1.returnItem();
18        System.out.println("\nFinal status : ");
19        b1.displayInfo();
20        d1.displayInfo();
21    }
```

INPUT

Your INPUT goes
here!

OUTPUT

```
Initial library items :
Title: The Alchemist
Author: paulo coelo
Status: Available
Pages: 208
Type: Book
-----
Title: Inception
Author: Nolan
Status: Available
```

QUESTIONS 19 / 20 completed

University

UNIVERSITY

Write a Java program to implement a basic university management system using inheritance. Create a parent class called Person with attributes such as name and age. Then, create child classes such as Student and Professor that inherit from the Person class and have their own

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.ArrayList;
2
3 public class University {
4     public static void main(String[] args) {
5         // Creating a Student object
6         Student student1 = new Student("suba", 20, "S123");
7         student1.enrollCourse("Java Programming");
8         student1.enrollCourse("Data Structures");
9
10        System.out.println("---- Student Details ----");
11        student1.displayInfo();
12        student1.displayCourses();
13
14        System.out.println();
15
16        // Creating a Professor object
17        Professor professor1 = new Professor("Dr. Ramesh", 45, "P67", "Computer Science");
18        professor1.teachClass("Object-Oriented Programming");
19
20        System.out.println("\n---- Professor Details ----");
21    }
```

INPUT

Your INPUT goes here!

OUTPUT

```
suba has enrolled in Java Programming
suba has enrolled in Data Structures
---- Student Details ----
Name: suba
Age: 20
Student ID: S123
suba's Enrolled Courses:
- Java Programming
- Data Structures
```

QUESTIONS 19 / 20 completed

Games

GAMES

Write a Java program to implement a basic game system using inheritance. Create a parent class called Game with attributes such as title and genre. Then, create child classes such as ActionGame and PuzzleGame that inherit from the Game class and have their own unique attributes.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         ActionGame ag = new ActionGame("Battle Warriors", "Action", "Guns and Swords")
4         PuzzleGame pg = new PuzzleGame("Mind Twister", "Puzzle", 50);
5
6         ag.displayInfo();
7         ag.play();
8
9         System.out.println();
10
11        pg.displayInfo();
12        pg.play();
13    }
14 }
15
16 class Game {
17     String title;
18     String genre;
19
20     Game(String title, String genre) {
21
```

INPUT

Your INPUT goes here!

OUTPUT

```
Title: Battle Warriors
Genre: Action
Battle Warriors is an action-packed game!
Weapon Type: Guns and Swords
Get ready for some intense combat!

Title: Mind Twister
Genre: Puzzle
Mind Twister is a brain-teasing puzzle game!
Total Levels: 50
```

QUESTIONS 19 / 20 completed

Resturant

RESTURANT

Write a Java program to implement a basic restaurant management system using inheritance. Create a parent class called Menu with attributes such as name and price. Then, create child classes such as Appetizer and Entree that inherit from the Menu class and have their own

PROGRAM EDITOR

Java

Run

Save

```

1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("--- Welcome to the Humanoid Bistro ---");
4
5         Appetizer starter = new Appetizer("Garlic Bread", 5.50, "Crispy and buttery");
6
7         Entree mainCourse = new Entree("Grilled Salmon", 15.99, "Fresh Atlantic salmon");
8
9         starter.order();
10        starter.serve();
11
12        System.out.println("-----");
13
14        mainCourse.order();
15        mainCourse.serve();
16
17        System.out.println("--- Thank you for dining with us! ---");
18    }
19 }
20
21
    
```

INPUT

Your INPUT goes here!

OUTPUT

```

--- Welcome to the Humanoid Bistro ---
Customer is ordering the appetizer: Garlic Bread
Serving Garlic Bread (Crispy and buttery). Enjoy your starter!
-----
Customer is ordering the entree: Grilled Salmon
Serving Grilled Salmon made with Fresh Atlantic salmon. Enjoy your main meal!
--- Thank you for dining with us! ---
    
```

QUESTIONS 19 / 20 completed

Forecasting

FORECASTING

Write a Java program to implement a basic weather forecasting system using inheritance. Create a parent class called Forecast with attributes such as date and temperature. Then, create child classes such as SunnyForecast and RainyForecast that inherit from the

PROGRAM EDITOR

Java

Run

Save

```

1 public class WeatherSystemSimulator {
2     public static void main(String[] args) {
3         SunnyForecast sunnyDay = new SunnyForecast("2023-10-25", 32.0);
4         sunnyDay.display();
5         sunnyDay.predict();
6
7         RainyForecast rainyDay = new RainyForecast("2023-10-26", 22.5);
8         rainyDay.display();
9         rainyDay.predict();
10    }
11 }
12 class Forecast {
13     String date;
14     double temperature;
15
16     public Forecast(String date, double temperature) {
17         this.date = date;
18         this.temperature = temperature;
19     }
20
21
    
```

INPUT

Your INPUT goes here!

QUESTIONS 19 / 20 completed

Rental

RENTAL

Write a Java program to implement a basic car rental system using inheritance. Create a parent class called Vehicle with attributes such as make and model. Then, create child classes such as Sedan and SUV that inherit from the Vehicle class and have their own unique attributes and

PROGRAM EDITOR

Java

Run

Save

```
1 public class MainRentalApp {
2     public static void main(String[] args) {
3         Vehicle sedan = new Sedan("Toyota", "Camry", "Medium Sedan");
4         Vehicle suv = new SUV("Honda", "CR-V", "Spacious SUV");
5         sedan.rent("2 days");
6         sedan.displayRentalDetails();
7         suv.rent("5 days");
8         suv.displayRentalDetails();
9         sedan.returnCar();
10        suv.returnCar();
11        sedan.displayRentalDetails();
12        suv.displayRentalDetails();
13    }
14 }
15 class Vehicle {
16     String make;
17     String model;
18     boolean isRented;
19     String rentalDuration;
20     public Vehicle(String make, String model) {
```

OUTPUT

```
Sedan perk: Medium Sedan
Rented: Sedan - Toyota Camry
Duration: 2 days
```

INPUT

Your INPUT goes here!

QUESTIONS 19 / 20 completed

Shape-2

SHAPE-2

Write a Java program to implement a basic shape hierarchy using polymorphism. Create a parent class called Shape with attributes such as area and perimeter. Then, create child classes such as Circle and Rectangle that inherit from the Shape class and have their own unique methods for

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Shape shape1 = new Circle(5);
4         Shape shape2 = new Rectangle(4, 6);
5         System.out.println("=== Circle ===");
6         System.out.println("Area: " + shape1.calculateArea());
7         System.out.println("Perimeter: " + shape1.calculatePerimeter());
8         System.out.println("\n=== Rectangle ===");
9         System.out.println("Area: " + shape2.calculateArea());
10        System.out.println("Perimeter: " + shape2.calculatePerimeter());
11    }
12 }
13 class Shape {
14     double area;
15     double perimeter;
16     public double calculateArea() {
17         return area;
18     }
19     public double calculatePerimeter() {
20         return perimeter;
21     }
22 }
```

OUTPUT

```
=== Circle ===
Area: 78.53981633974483
Perimeter: 31.41592653589793
```

INPUT

Your INPUT goes here!

QUESTIONS 19 / 20 completed

Draw

DRAW

Write a Java program to demonstrate polymorphism with interfaces. Create an interface called Drawable with a method called draw(). Then, create classes such as Circle and Square that implement the Drawable interface and have their own unique implementations of draw().

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Drawable myCircle = new Circle();
4         Drawable mySquare = new Square();
5
6         System.out.println("--- Demonstrating Polymorphism ---");
7         myCircle.draw();
8         mySquare.draw();
9     }
10 }
11 interface Drawable {
12     void draw();
13 }
14 class Circle implements Drawable {
15     @Override
16     public void draw() {
17         System.out.println("Drawing a Circle: o");
18     }
19 }
20 class Square implements Drawable {
21     @Override
```

INPUT

Your INPUT goes here!

OUTPUT

```
--- Demonstrating Polymorphism ---
Drawing a Circle: o
Drawing a Square: []
```

QUESTIONS 19 / 20 completed

Overloading

OVERLOADING

Write a Java program to demonstrate method overloading with variable arguments. Create a method called sum() that takes in a variable number of integers and returns their sum. Then, create an overloaded version of the sum() method that takes in a variable number of doubles.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         int intSum = sum(10, 20, 30, 40);
4         System.out.println("Sum of integers: " + intSum);
5
6         double doubleSum = sum(10.5, 20.2, 30.3);
7         System.out.println("Sum of doubles: " + doubleSum);
8         System.out.println("Sum of nothing: " + sum());
9     }
10
11     public static int sum(int... numbers) {
12         int total = 0;
13         for (int number : numbers) {
14             total += number;
15         }
16         return total;
17     }
18
19     public static double sum(double... numbers) {
20         double total = 0.0;
21         for (double number : numbers) {
```

INPUT

Your INPUT goes here!

OUTPUT

```
Sum of integers: 100
Sum of doubles: 61.0
Sum of nothing: 0
```

QUESTIONS 19 / 20 completed

Overriding

OVERRIDING

Write a Java program to demonstrate method overriding with a simple calculator. Create a parent class called Calculator with methods such as add(), subtract(), multiply(), and divide(). Then, create a child class called ScientificCalculator that overrides the

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Calculator basicCalc = new Calculator();
4
5         ScientificCalculator sciCalc = new ScientificCalculator();
6
7         System.out.println("--- Basic Calculator Operations ---");
8         System.out.println("Addition: " + basicCalc.add(10, 5));
9         System.out.println("Subtraction: " + basicCalc.subtract(10, 5));
10        System.out.println("Multiplication: " + basicCalc.multiply(10, 5));
11        System.out.println("Division: " + basicCalc.divide(10, 5));
12
13        System.out.println("\n--- Scientific Calculator Operations ---");
14        System.out.println("Addition: " + sciCalc.add(10, 5)); // Inherited
15        System.out.println("Subtraction: " + sciCalc.subtract(10, 5)); // Inherited
16        System.out.println("Scientific Multiplication (x10 factor): " + sciCalc.multiply(10, 5));
17        System.out.println("Division: " + sciCalc.divide(10, 5)); // Inherited
18    }
19 }
20
21
```

INPUT

Your INPUT goes
here!

OUTPUT

```
--- Basic Calculator Operations ---
Addition: 15.0
Subtraction: 5.0
Multiplication: 50.0
Division: 2.0

--- Scientific Calculator Operations ---
Addition: 15.0
Subtraction: 5.0
Scientific Multiplication (x10 factor): 500.0
```

QUESTIONS 19 / 20 completed

Person

PERSON

Write a Java program to create a class called Person with a constructor that takes in a name and age, and a method to print out the person's name and age.

PROGRAM EDITOR

Java

Run

Save

```
1 public class PersonApp {
2     public static void main(String[] args) {
3         Person p = new Person("suba", 20);
4         p.print();
5     }
6 }
7 class Person {
8     private String name;
9     private int age;
10    public Person(String name, int age) {
11        this.name = name;
12        this.age = age;
13    }
14    public void print() {
15        System.out.println("Name: " + name);
16        System.out.println("Age: " + age);
17    }
18 }
```

OUTPUT

```
Name: suba
Age: 20
```

INPUT

Your INPUT goes here!

QUESTIONS 19 / 20 completed

Rect-Const

RECT-CONST

Write a Java program to create a class called Rectangle with a constructor that takes in the length and width of the rectangle, and a method to calculate and return the area of the rectangle.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main{
2     public static void main(String args[]){
3         Rectangle r=new Rectangle(5.0,6.0);
4         r.area();
5     }
6 }
7 class Rectangle{
8     double length;
9     double width;
10    Rectangle(double length,double width){
11        this.length=length;
12        this.width=width;
13    }
14    void area(){
15        System.out.println("Area of reactangle : "+(length*width));
16    }
17 }
```

OUTPUT

```
Area of reactangle : 30.0
```

QUESTIONS 19 / 20 completed

Bank-Const

BANK-CONST

Write a Java program to create a class called BankAccount with a constructor that takes in an account number and an initial balance, and methods to deposit and withdraw money from the account.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String accNum = sc.nextLine();
6         double initialBalance = sc.nextDouble();
7         BankAccount Ac = new BankAccount(accNum, initialBalance);
8         Ac.display();
9         double depositAmount = sc.nextDouble();
10        Ac.deposit(depositAmount);
11        double withdrawAmount = sc.nextDouble();
12        Ac.withdraw(withdrawAmount);
13        Ac.display();
14    }
15 }
16 class BankAccount {
17     String accountNumber;
18     double balance;
19     public BankAccount(String accountNumber, double initialBalance) {
20         this.accountNumber = accountNumber;
21         this.balance = initialBalance;
```

INPUT

```
123456789
5000
2000
3000
```

OUTPUT

```
Account Number: 123456789
Current Balance: $5000.0
Successfully deposited: $2000.0
Successfully withdrew: $3000.0
Account Number: 123456789
Current Balance: $4000.0
```

QUESTIONS 19 / 20 completed

Car-Const

CAR-CONST

Write a Java program to create a class called Car with a constructor that takes in the make, model, and year of the car, and a method to print out the car's make, model, and year.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Car myCar = new Car("Tesla", "Model S", 2023);
4         myCar.displayCarDetails();
5     }
6 }
7
8 class Car {
9     private String make;
10    private String model;
11    private int year;
12    public Car(String make, String model, int year) {
13        this.make = make;
14        this.model = model;
15        this.year = year;
16    }
17    public void displayCarDetails() {
18        System.out.println("Car Details: " + year + " " + make + " " + model);
19    }
20 }
```

OUTPUT

Car Details: 2023 Tesla Model S

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Student-Const

STUDENT-CONST

Write a Java program to create a class called Student with a constructor that takes in a name, ID number, and a list of grades, and methods to calculate and return the student's average grade and letter grade.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.List;
2 import java.util.Arrays;
3
4 public class Main {
5     public static void main(String[] args) {
6         Student st = new Student("mosu", 165, Arrays.asList(95.0, 90.0, 89.0, 99.0));
7         System.out.println("Student Name: " + st.getName());
8         System.out.println("Student ID: " + st.getId());
9         System.out.println("Average Grade: " + st.calculateAverage());
10        System.out.println("Letter Grade: " + st.getLetterGrade());
11    }
12 }
13
14 class Student {
15     private String name;
16     private int id;
17     private List<Double> grades;
18     public Student(String name, int id, List<Double> grades) {
19         this.name = name;
20         this.id = id;
21         this.grades = grades;
```

INPUT

Your INPUT goes
here!

OUTPUT

```
Student Name: mosu
Student ID: 165
Average Grade: 93.25
Letter Grade: A
```